Spring Boot Study Guide — Introduction to Spring Boot

Overview

Spring Boot is a Java framework that simplifies the creation of standalone, production-oriented applications built on the Spring ecosystem. It uses sensible defaults and auto-configuration to reduce boilerplate setup.

Getting Started

A typical Spring Boot application uses the @SpringBootApplication annotation and starts through SpringApplication.run(). Spring Boot can embed a web server such as Tomcat, making it possible to run a web application as an executable JAR.

Core Benefits

Spring Boot accelerates application setup, integrates with the Spring ecosystem, supports externalized configuration, and offers production monitoring through Spring Boot Actuator.

Spring Boot Study Guide — REST APIs and Web Controllers

REST Endpoints

Spring MVC supports HTTP APIs using @RestController, @RequestMapping, @GetMapping, @PostMapping, @PutMapping, and @DeleteMapping. Controllers receive requests, validate inputs, and delegate business logic to services.

Request and Response

@RequestBody maps JSON request bodies into Java objects. @PathVariable reads URL path parameters, while @RequestParam reads query parameters. ResponseEntity can control HTTP status codes, headers, and response bodies.

API Design

Well-designed APIs use clear resource paths, meaningful status codes, input validation, pagination for collections, and consistent error responses. Business logic generally belongs in a service layer rather than controllers.

Spring Boot Study Guide — Dependency Injection and Application Layers

Inversion of Control

Spring manages application objects called beans. Dependency injection supplies required collaborators to a class instead of making the class construct them directly. Constructor injection is commonly preferred because dependencies are explicit and easier to test.

Stereotype Annotations

@Component registers a general-purpose bean; @Service communicates business-service intent; @Repository identifies persistence components and enables persistence-related exception translation. @RestController handles web API requests.

Layered Architecture

A common application separates controller, service, and repository responsibilities. The controller handles HTTP, the service enforces business rules, and the repository accesses stored data. This separation improves maintainability and testing.

Spring Boot Study Guide — Spring Data JPA and Transactions

Repositories

Spring Data JPA simplifies relational database access. A repository extending JpaRepository can provide CRUD methods, pagination, and sorting. Entities are mapped to database tables using JPA annotations such as @Entity, @Id, and @GeneratedValue.

ORM and Queries

Hibernate commonly implements JPA in Spring Boot projects. Developers can define derived query methods, JPQL queries, or native SQL where needed. Fetch strategies and the N+1 query problem should be considered when loading related entities.

Transactions

@Transactional defines a transactional boundary for database operations. Transactions help ensure that related changes succeed or fail together. Correct transaction boundaries are especially important for workflows such as order placement and inventory updates.

Spring Boot Study Guide — Testing, Configuration, and Operations

Testing

JUnit 5 and Mockito are commonly used for unit tests. @WebMvcTest can focus on controller behavior, while @SpringBootTest loads a wider application context for integration testing. Tests should cover success cases, validation failures, and error handling.

Configuration and Security

application.properties or application.yml holds configuration. Environment variables and profiles support different environments. Secrets should not be hardcoded. Spring Security can protect endpoints and support authentication and authorization.

Deployment and Monitoring

Spring Boot applications can be packaged as executable JARs or container images. Actuator provides operational endpoints such as health and metrics. Production systems should include structured logging, observability, database migration practices, and appropriate resource limits.